

Understanding Fine-Tuning



When and Why to Customize Model Weights

Module 7: Model Customization • Section 7.1

Nunnari Academy

Generative AI & Agentic AI Program

What We Will Cover

1

What is Fine-Tuning

Definition, core concept, and how it differs from training from scratch

2

How Fine-Tuning Works

The end-to-end process from data preparation to deployment

3

When to Fine-Tune

Key signals that indicate fine-tuning is the right choice

4

Fine-Tuning vs RAG vs Prompting

A clear decision framework for choosing the right approach

What is Fine-Tuning?

1 What is Fine-Tuning?

Core Concept: Fine-tuning adjusts a pre-trained model's internal weights using a domain-specific dataset, producing a specialized version that retains general capabilities while excelling at targeted tasks.

Key Points

Builds on existing knowledge: Starts from a pre-trained foundation model, not from scratch

Updates model weights: Additional training on curated input-output examples

Permanent internalization: The model learns new patterns, tone, or domain expertise

Reduces prompt dependency: Fine-tuned models need shorter prompts, produce more consistent results

Analogy

A pre-trained model is like a medical school graduate — broad knowledge, but not yet specialized.

Fine-tuning is the residency — focused training that turns a generalist into a specialist.

How Fine-Tuning Works

2 The Fine-Tuning Process

Process: Five steps from raw data to a deployed, specialized model.

- 1 **Prepare Your Dataset** — Curate 500–10,000+ high-quality input-output pairs that represent the desired behavior
- 2 **Choose a Base Model** — Select a pre-trained model (e.g. QWEN , LLaMA, Mistral) that fits your size and cost constraints
- 3 **Configure Training** — Set hyperparameters: learning rate, epochs, batch size — typically 1–5 epochs to avoid overfitting
- 4 **Train the Model** — Run supervised fine-tuning (SFT); the model adjusts weights to minimize loss on your dataset
- 5 **Evaluate & Deploy** — Test on held-out data, measure accuracy and regression, then deploy the fine-tuned model

When to Fine-Tune

3

When to Fine-Tune

Fine-Tune When...

- ✓ You need consistent output format or style across thousands of calls
- ✓ Prompt engineering alone produces unreliable or inconsistent results
- ✓ You have high-quality labeled data specific to your task
- ✓ Latency matters — fine-tuned models skip lengthy system prompts
- ✓ The behavior you need doesn't exist in the base model's training

Do NOT Fine-Tune When...

- ✗ Your knowledge base changes frequently — use RAG instead
- ✗ You're still prototyping — start with prompt engineering
- ✗ You lack sufficient quality training data
- ✗ The base model already performs well with a good prompt

Rule of thumb: Start with prompting, add RAG for knowledge, fine-tune only for persistent behavioral changes.

Fine-Tuning vs RAG vs Prompt Engineering

Choosing the Right Approach

Criteria	Prompt Engineering	RAG	Fine-Tuning
Setup Effort	Low (minutes–hours)	Medium (days–weeks)	High (weeks–months)
Data Required	None to few examples	Document corpus	500–10,000+ labeled pairs
Knowledge Updates	Instant (edit prompt)	Fast (update index)	Slow (retrain model)
Best For	Formatting, tone, simple tasks	Factual Q&A, dynamic knowledge	Consistent style, specialized behavior
Latency Impact	Minimal	Moderate (retrieval step)	Minimal (no extra lookup)
Cost	Token costs only	Infra + token costs	Training + hosting costs

These approaches are complementary — production systems often combine all three together.

Key Takeaways

- 1 **Fine-tuning adapts model weights** — It creates a specialized version of a general-purpose model
- 2 **Data quality is everything** — The output is only as good as the training examples you provide
- 3 **It reduces prompt dependency** — Fine-tuned models need shorter prompts and produce more consistent results
- 4 **It's not always the answer** — Use prompt engineering for quick iteration, RAG for dynamic knowledge
- 5 **Combine all three** — The most effective systems layer prompting, retrieval, and fine-tuning together

Fine-tune when you need persistent behavioral changes that prompts alone cannot reliably achieve.

Types of Fine-Tuning



Full, LoRA & QLoRA – Adapting LLMs to Your Needs

Module: Fine-Tuning Large Language Models

Nunnari Academy

Generative AI & Agentic AI Program

Three Approaches to Fine-Tuning

1

Full Fine-Tuning

Updates every parameter in the model. Maximum flexibility, maximum cost.

2

LoRA

Injects small trainable matrices while freezing the original weights.

3

QLoRA

Combines LoRA with 4-bit quantization to slash memory usage further.

Full
Fine-Tuning

1 Full Fine-Tuning

Core Principle: Full fine-tuning updates all model parameters during training, giving the model complete freedom to adapt to new data.

Key Characteristics

Trainable Parameters: 100% of the model weights are updated during backpropagation.

Learning Capacity: Can learn entirely new behaviors, domains, or output styles.

Hardware Demand: Requires GPU memory proportional to full model size (weights + gradients + optimizer states).

Risk of Forgetting: Higher chance of catastrophic forgetting — the model may lose general capabilities.

When to Use

1. You have a large, high-quality domain-specific dataset.
2. The target task diverges significantly from the base model's pretraining distribution.
3. Compute budget and infrastructure are not constraints.
4. You need the absolute best task-specific performance.



Resource & Trade-Off Profile

Aspect	Detail
Memory (7B model)	~60–80 GB VRAM (FP16 weights + Adam optimizer states)
Training Speed	Slowest — every layer participates in gradient computation
Storage per Task	One full model copy per task (≈ 14 GB for a 7B model in FP16)
Best For	Domain shift, new languages, safety realignment
Watch Out For	Overfitting on small datasets; catastrophic forgetting of base knowledge

Full fine-tuning is the gold standard for performance — but it pays the full price in compute and memory.

LoRA

(Low-Rank Adaptation)

2

LoRA – Low-Rank Adaptation

Core Idea: LoRA freezes all original weights and injects small, trainable low-rank matrices (A and B) into each transformer layer. Only these matrices are trained.

The Process

- 1 Freeze the pretrained weight matrix W .
- 2 Add two small matrices: A ($d \times r$) and B ($r \times d$), where $r \ll d$.
- 3 Effective weight becomes $W + BA$ during forward pass.
- 4 Only A and B receive gradient updates during training.
- 5 At inference, merge BA back into W – zero added latency.

Why It Works

- Weight updates during fine-tuning tend to be low-rank in practice – most change is captured in a small subspace.
- A rank of $r = 8$ to 64 is usually sufficient, meaning LoRA trains less than 1% of total parameters.
- The original model stays untouched, so you can swap LoRA adapters for different tasks on the same base model.



Resource & Trade-Off Profile

Aspect	Detail
Memory (7B model)	~16–20 GB VRAM (frozen weights in FP16 + small adapter gradients)
Trainable Params	Typically 0.1%–1% of total model parameters
Training Speed	2–3× faster than full fine-tuning
Storage per Task	Only the adapter weights (~5–50 MB per task)
Inference Latency	Zero overhead after merging adapters into base weights
Best For	Task adaptation, style transfer, multi-task serving

LoRA delivers 90–99% of full fine-tuning quality at a fraction of the cost — the default choice for most use cases.

QLoRA

(Quantized LoRA)

3 QLoRA – Quantized Low-Rank Adaptation

Core Idea: QLoRA quantizes the frozen base model to 4-bit precision and applies LoRA adapters on top, enabling fine-tuning of very large models on consumer-grade GPUs.

Three Key Innovations

1

4-bit NormalFloat (NF4)

A data type optimized for normally distributed neural network weights – more information-dense than standard INT4.

2

Double Quantization

Quantizes the quantization constants themselves, saving an additional ~0.4 bits per parameter.

3

Paged Optimizers

Uses CPU RAM as overflow when GPU memory spikes, preventing out-of-memory crashes.

Practical Impact

→ A 65B-parameter model fits on a single 48 GB GPU for fine-tuning.

→ A 7B model can be fine-tuned on a single 12–16 GB consumer GPU.

→ Quality loss from 4-bit quantization is minimal – matches 16-bit LoRA benchmarks.

→ All computation uses BFloat16 – only stored weights are in 4-bit.



Resource & Trade-Off Profile

Aspect	Detail
Memory (7B model)	~6–10 GB VRAM (4-bit weights + LoRA adapter gradients)
Trainable Params	Same as LoRA: 0.1%–1% of total parameters
Training Speed	Comparable to LoRA; slight overhead from dequantization
Storage per Task	4-bit base model (~3.5 GB for 7B) + adapter (~5–50 MB)
Best For	Large models on limited hardware, rapid prototyping, accessibility
Watch Out For	Slightly more complex setup; some edge-case quality loss on reasoning tasks

QLoRA democratizes fine-tuning — making models that once required data-center GPUs trainable on a single consumer card.

Comparison & Summary

Full vs. LoRA vs. QLoRA – At a Glance

Dimension	Full Fine-Tuning	LoRA	QLoRA
Params Trained	100%	0.1–1%	0.1–1%
VRAM (7B)	60–80 GB	16–20 GB	6–10 GB
Training Speed	Baseline	2–3× faster	~2–3× faster
Task Storage	Full model copy	Adapter only (~MB)	4-bit model + adapter
Quality	Highest ceiling	Near-full quality	Near-full quality
Multi-Task	Separate models	Swap adapters	Swap adapters
Hardware	Multi-GPU / A100+	Single mid-range GPU	Single consumer GPU

Start with QLoRA for prototyping, graduate to LoRA for production, reserve full fine-tuning for specialized domains.

Key Takeaways

1

Full Fine-Tuning: Updates every weight — maximum power, maximum cost, risk of catastrophic forgetting.

2

LoRA: Freezes the base model and trains tiny low-rank adapters — the sweet spot of quality and efficiency.

3

QLoRA: Adds 4-bit quantization to LoRA — makes large-model fine-tuning accessible on consumer hardware.

4

Choose by Constraint: Your compute budget, dataset size, and deployment needs determine the right approach.

5

Adapters Are the Future: LoRA/QLoRA enable multi-task serving from a single base model with swappable, lightweight adapters.

Efficiency and quality are no longer at odds — LoRA and QLoRA prove you can adapt billion-parameter models without billion-dollar budgets.

Memory Architecture & Quantization for LLMs



How Numbers Are Stored, and Why It Matters for Fine-Tuning

Module 7: Model Customization • Section 7.5

Nunnari Academy

Generative AI & Agentic AI Program

What We Will Cover

1

Why Memory Matters

Billions of parameters —
how you store each one
determines GPU feasibility

2

Data Types Explained

FP32, FP16, and BF16 —
the three ways numbers
are stored in deep learning

3

Quantization by Binning

The simple idea of
assigning values to bins
to shrink model size

4

8-Bit & 4-Bit in Practice

How BitsAndBytes applies
quantization with NF4
and Int8

Why Memory Matters

1 The Memory Problem

Key Insight: Every parameter in an LLM is a number stored in memory. A 7B parameter model in FP32 takes ~28 GB of GPU RAM — more than most consumer GPUs have.

```
model_size_MB = ( num_params * bits_per_param / 8 ) / 1,000,000
```

360M params × FP32 (32 bits) = ~1.4 GB

360M params × FP16 (16 bits) = ~720 MB

7B params × FP32 (32 bits) = ~28 GB

7B params × FP16 (16 bits) = ~14 GB

7B params × INT4 (4 bits) = ~3.5 GB

By changing how we store each parameter, we can shrink a 28 GB model to 3.5 GB — that's the power of quantization.

Data Types Explained

2a FP32: Full Precision (32-bit)

Definition: FP32 uses 32 bits per number — 1 sign bit, 8 exponent bits, and 23 mantissa bits — giving excellent precision and range. The gold standard of deep learning.

1 Sign

8 Exponent Bits

23 Mantissa Bits

Precision: Accurate to ~7 decimal places (1e-08)

Range: Approximately $\pm 3.4 \times 10^{38}$ — enormous

Memory: 4 bytes per parameter

Trade-off: Extremely accurate, but a 7B model needs ~28 GB just for weights

2b

FP16: Half Precision (16-bit)

Definition: FP16 uses 16 bits — 1 sign bit, 5 exponent bits, and 10 mantissa bits. Halves memory but comes with a significantly narrower range.

1 Sign

5 Exponent

10 Mantissa

Key Facts

✓ Precision: ~4 decimal places (1e-04)

✓ Range: $\pm 65,504$ only

✓ Memory: 2 bytes (50% savings over FP32)

✓ Smallest normal: 6.1×10^{-5}

The Danger — Living on the Edge

✗ Overflow: values $> 65,504$ become infinity (inf)

✗ Underflow: values $< \sim 6 \times 10^{-8}$ become zero

✗ These cause NaN losses and crash training

2c

BF16: The Brain Float (16-bit)

Definition: BF16 was created by Google Brain to fix FP16's problems. It uses 1 sign bit, 8 exponent bits (same as FP32), and only 7 mantissa bits — trading a small amount of precision for the full range of FP32.

1 Sign

8 Exponent (same as FP32)

7 Mantissa

✓ **Same range as FP32:** No training crashes from overflow or underflow

✓ **Same memory as FP16:** You still get the 50% savings over FP32 (2 bytes per param)

✓ **Negligible precision loss:** 3 vs 4 decimal places — barely matters for LLM training

✓ **The recommended dtype:** Use BF16 for fine-tuning on modern GPUs (A100, H100, RTX 3090+)

BF16 gives you FP32's range at FP16's memory cost — the best of both worlds for training.

Data Types Comparison

Property	FP32	FP16	BF16
Total Bits	32	16	16
Exponent Bits	8	5	8
Mantissa Bits	23	10	7
Precision	$\sim 1e-08$	$\sim 1e-04$	$\sim 1e-03$
Max Value	$\sim 3.4e+38$	$\sim 6.5e+04$	$\sim 3.4e+38$
Min Normal	$\sim 1.2e-38$	$\sim 6.1e-05$	$\sim 1.2e-38$
Bytes / Param	4	2	2
Overflow Risk	None	High	None

BF16 combines FP32's range with FP16's memory efficiency – the recommended choice for fine-tuning.

Quantization by Binning

3 How Quantization Works: Binning

Core Idea: Quantization is binning. Take the full range of FP32 values, divide into a fixed number of bins, and replace each value with its bin index. Fewer bins = fewer bits = smaller model.

1 **Define the Range** — Find the min and max of the FP32 weights (e.g., -0.2 to 0.2)

2 **Divide into Bins** — Split that range into N equal-width bins, where $N = 2^{\text{bits}}$ (e.g., 4 bits = 16 bins)

3 **Assign Bin Indexes** — Each weight gets replaced by the index of the bin it falls into — that index IS the quantized value

Why It Works: Linear layers in LLMs have weights concentrated in a narrow, zero-centered range with very few outliers. This distribution is perfect for binning with minimal precision loss.

Bits vs Bins vs Error

```
n_bins = 2 ^ n_bits | approx_value = bin_index * bin_width + first_bin
```

2-bit

4 bins | RMSE: 0.0615 | Very crude approximation

4-bit

16 bins | RMSE: 0.0152 | Decent — usable for most tasks

8-bit

256 bins | RMSE: 0.0010 | Excellent — near-lossless

16-bit

65,536 bins | RMSE: 0.0001 | Near-perfect approximation

You can never recover exact original values — only approximations. But with 4+ bits, model quality barely suffers.

8-Bit & 4-Bit Quantization

4a 8-Bit Quantization (LLM.int8())

How It Works: LLM.int8() quantizes most weights to 8-bit integers but handles outlier values separately in 16-bit — preserving model quality with no performance degradation.

1 Quantization: Most features quantized to 8-bit using vector-wise quantization

2 Outlier Handling: Values above threshold (6.0) kept in 16-bit precision

3 Combination: 8-bit and 16-bit outputs combined for the final result

4 Selective Targeting: Only large linear layers in decoder blocks are quantized — embeddings and heads stay in FP16

Result: Model shrinks to ~25% of original FP32 size with no performance degradation.

4b

4-Bit Quantization (QLoRA)

How It Works: Shrinks models to ~12.5% of FP32 size. Choose between FP4 (standard) or NF4 (NormalFloat4) — designed for the normally distributed weights found in LLMs.

Recommended Configuration

```
BitsAndBytesConfig(  
    load_in_4bit=True,  
    bnb_4bit_quant_type="nf4",  
    bnb_4bit_use_double_quant=True,  
    bnb_4bit_compute_dtype=torch.bfloat16  
)
```

Data Type Flow in Quantized Models

1. Weights stored as 4-bit integers (saves space)

2. At compute: dequantized to BF16/FP32

3. Inputs cast to match compute dtype

4. Multiply inputs × weights in compute dtype

5. Output cast back to torch_dtype for next layer

4-bit NF4 + double quantization + BF16 compute is the gold standard for QLoRA fine-tuning.

Key Takeaways

- 1 **Memory determines feasibility** — Every parameter is a number — how you store it determines whether the model fits on your GPU
- 2 **BF16 is the training sweet spot** — FP32's range at FP16's memory cost, with no overflow crashes
- 3 **Quantization is just binning** — Divide the value range into bins, replace values with bin indexes, shrink the model
- 4 **More bits = less error** — 4-bit gives decent quality (RMSE 0.015), 8-bit is excellent (RMSE 0.001)
- 5 **NF4 + BF16 is the standard** — 4-bit NormalFloat quantization with BF16 compute is the go-to config for QLoRA

Understanding data types and quantization is the foundation that makes fine-tuning large models on accessible hardware possible.

PARAMETER EFFICIENCY



Axis - 1 : Parameter Efficiency outlook on SFT.

Module 7: Model Customization • Section 7.3

Nunnari Academy

Generative AI & Agentic AI Program

1 Full Fine-Tuning

Core Principle: Full fine-tuning updates all model parameters during training, giving the model complete freedom to adapt to new data.

Key Characteristics

Trainable Parameters: 100% of the model weights are updated during backpropagation.

Learning Capacity: Can learn entirely new behaviors, domains, or output styles.

Hardware Demand: Requires GPU memory proportional to full model size (weights + gradients + optimizer states).

Risk of Forgetting: Higher chance of catastrophic forgetting — the model may lose general capabilities.

When to Use

1. You have a large, high-quality domain-specific dataset.
2. The target task diverges significantly from the base model's pretraining distribution.
3. Compute budget and infrastructure are not constraints.
4. You need the absolute best task-specific performance.

2

LoRA – Low-Rank Adaptation

Core Idea: LoRA freezes all original weights and injects small, trainable low-rank matrices (A and B) into each transformer layer. Only these matrices are trained.

The Process

- 1 Freeze the pretrained weight matrix W .
- 2 Add two small matrices: A ($d \times r$) and B ($r \times d$), where $r \ll d$.
- 3 Effective weight becomes $W + BA$ during forward pass.
- 4 Only A and B receive gradient updates during training.
- 5 At inference, merge BA back into W – zero added latency.

Why It Works

- Weight updates during fine-tuning tend to be low-rank in practice – most change is captured in a small subspace.
- A rank of $r = 8$ to 64 is usually sufficient, meaning LoRA trains less than 1% of total parameters.
- The original model stays untouched, so you can swap LoRA adapters for different tasks on the same base model.

3 QLoRA – Quantized Low-Rank Adaptation

Core Idea: QLoRA quantizes the frozen base model to 4-bit precision and applies LoRA adapters on top, enabling fine-tuning of very large models on consumer-grade GPUs.

Three Key Innovations

1

4-bit NormalFloat (NF4)

A data type optimized for normally distributed neural network weights – more information-dense than standard INT4.

2

Double Quantization

Quantizes the quantization constants themselves, saving an additional ~0.4 bits per parameter.

3

Paged Optimizers

Uses CPU RAM as overflow when GPU memory spikes, preventing out-of-memory crashes.

Practical Impact

→ A 65B-parameter model fits on a single 48 GB GPU for fine-tuning.

→ A 7B model can be fine-tuned on a single 12–16 GB consumer GPU.

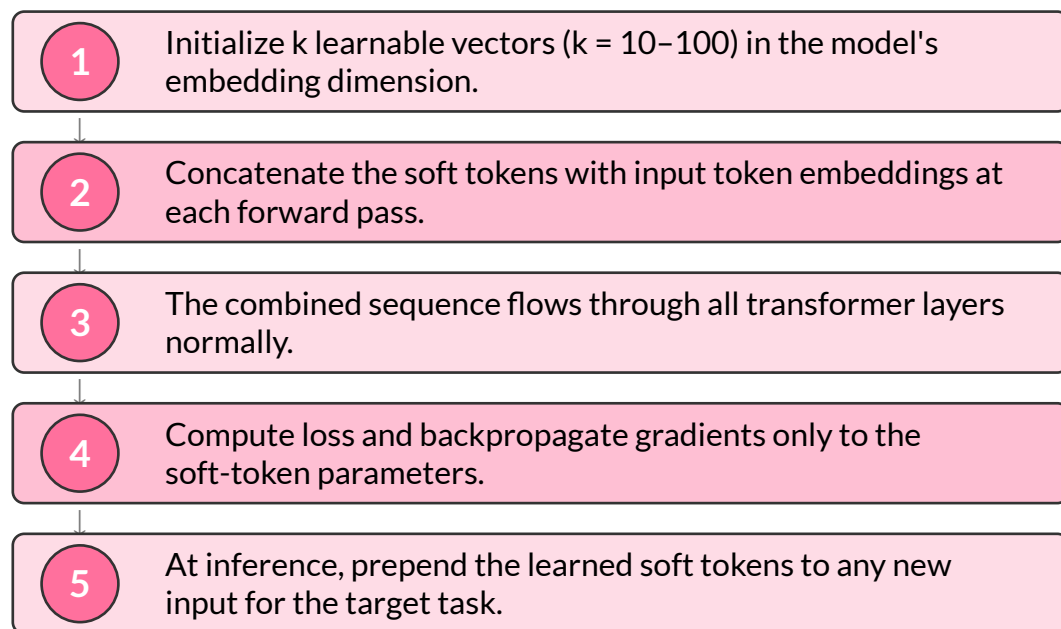
→ Quality loss from 4-bit quantization is minimal – matches 16-bit LoRA benchmarks.

→ All computation uses BFloat16 – only stored weights are in 4-bit.

4 Prompt Tuning

Definition: Prompt tuning prepends a sequence of trainable embedding vectors to the input layer only. These soft tokens are concatenated with the real input embeddings before the first transformer layer.

The Process



Key Properties

- Trainable parameters: $k \times d$. For $k=20$ on a 7B model ($d=4096$), that is just ~80K parameters.
- Soft tokens influence all layers indirectly through attention, but are only injected at the input layer.
- As model size increases, prompt tuning performance approaches full fine-tuning – at 11B+ scale, the gap nearly closes.
- Originally proposed by Lester et al. (2021) as a simplified alternative to prefix tuning.

5 Prefix Tuning

Definition: Prepends a sequence of learnable virtual tokens (the “prefix”) to the input at every transformer layer. The model attends to these tokens as if they were part of the input.

How It Adapts

Adds continuous trainable embeddings at each layer’s key-value input

Params Trained

0.1–1% — controlled by prefix length (typically 10–100 virtual tokens)

Where It Modifies

Prepended to every transformer layer’s attention computation

Inference Impact

Slight latency increase from extra tokens in attention computation

Key Strength: No architectural changes to the model — purely additive, non-invasive

6

IA3 - Infused Adapter by Inhibiting and Amplifying Inner Activations.

Definition: Learns three vectors that rescale Key, Value, and feed-forward activations. No new layers or matrices — just element-wise multiplication with learned scaling vectors.

How It Adapts

Element-wise rescaling of existing activations using learned vectors

Params Trained

0.01-0.05% — the fewest of any PEFT method

Where It Modifies

Key projections, Value projections, and feed-forward activations

Inference Impact

Zero added latency — scaling vectors merge into existing weights

Key Strength: Extreme parameter efficiency — 10x fewer parameters than LoRA with competitive results

7

Bottleneck Adapters

Definition: Inserts small bottleneck layers (down-project → nonlinearity → up-project) between existing transformer sub-layers. Data flows through the adapter, gets transformed, then continues.

How It Adapts

Sequential insertion of small feed-forward networks inside each block

Params Trained

0.5–5% – controlled by the bottleneck dimension

Where It Modifies

Between the attention and feed-forward sub-layers in each block

Inference Impact

Slight latency increase from extra sequential computation

Key Strength: Well-studied, strong theoretical foundation, composable across tasks

Head-to-Head Comparison

Criteria	LoRA	QLoRA	Prefix Tuning	Adapters	IA3
Params Trained	0.1–1%	0.1–1%	0.1–1%	0.5–5%	0.01–0.05%
Memory Savings	High	Very High	High	Moderate	Very High
Inference Latency	None (merged)	None (merged)	Slight increase	Slight increase	None (merged)
Performance	Near full FT	Near full FT	Good	Good	Competitive
Ease of Use	Very Easy	Easy	Moderate	Moderate	Easy
Best For	General tasks	Low-memory setups	Generation tasks	Multi-task learning	Few-shot tasks

Head-to-Head Comparison

Dimension	Prompt Tuning	Prefix Tuning	LoRA (reference)
Injection Point	Input embedding only	Key-value at every layer	Weight matrices at target layers
Trainable Params	<0.01%	0.1–1%	0.1–1%
Quality ($\leq 10B$)	Moderate	Good	Very Good
Quality ($> 10B$)	Very Good	Very Good	Very Good
Inference Overhead	Longer input sequence	Longer KV context per layer	Zero (after merging)
Multi-Task	Swap prompt vectors	Swap prefix vectors	Swap/merge adapters
Best For	Very large models, extreme param budget	Small-to-mid models, generation tasks	General-purpose fine-tuning

LoRA remains the default for most tasks — but soft-prompt methods shine when parameter count or multi-task switching is the primary constraint.

1. Quantization $\rightarrow$ Config
2. Tokenization $\rightarrow$ Config
3. Model $\rightarrow$ Load
- 4) Test

5) Dataset $\rightarrow$ what kind of / Download

6) Dataset formatting $\rightarrow$ tokenization $\langle \text{BTR} \rangle \langle \text{END} \rangle$

7) SFT Trainer $\rightarrow$ LoRA, QLoRA,
Pref. Pr

8) SFT Config $\rightarrow$ (How epochs / steps)

9) Trainer.Train()

10) Eval / Test

PREFERENCE OPTIMIZATION



Axis - 2 : Training Objective outlook on SFT.

Module 7: Model Customization • Section 7.3

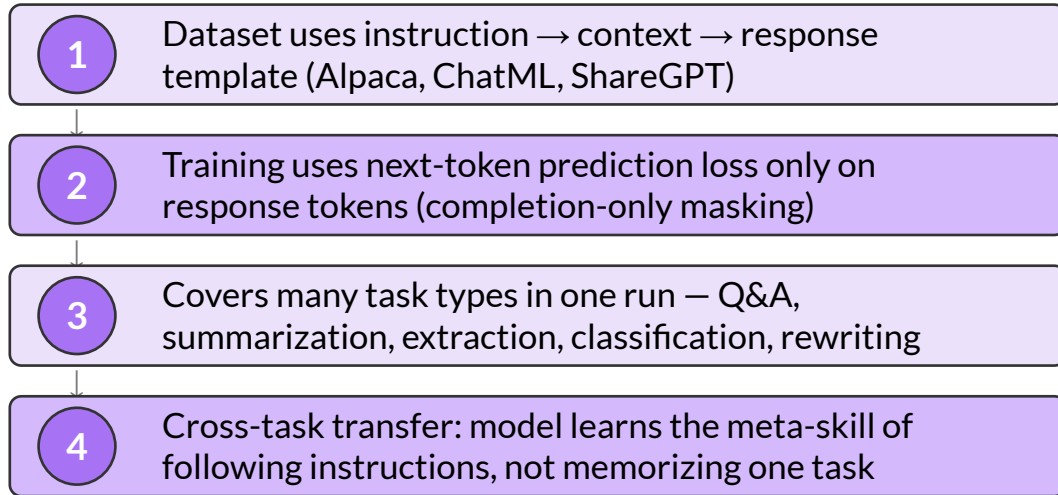
Nunnari Academy

Generative AI & Agentic AI Program

1 Instruction Tuning

Core Principle: Instruction tuning trains the model on diverse prompt-response pairs framed as explicit instructions, improving its ability to follow directions and generalize across tasks.

How It Works



Practical Details

- **Template matters:** Format at training must match inference — mismatched templates cause poor performance.
- **Quality over quantity:** 1,000 diverse, high-quality pairs often outperform 50,000 noisy ones.
- **Stage 2 in pipeline:** Best after continued pre-training. CPT injects knowledge; IT teaches how to use it.
- **Reduces overhead:** One instruction-tuned medical model can answer, summarize, and extract — no separate models.
- **Tooling:** TRL SFTTrainer, Unsloth FastLanguageModel, Axolotl, LLaMA-Factory.

Instruction tuning teaches a model how to respond — turning raw language ability into reliable, task-following behavior.

2

Direct Preference Optimization (DPO)

Core Idea: DPO trains the model to prefer chosen responses over rejected ones using a contrastive loss — achieving alignment without a separate reward model or RL loop.

How It Works

1

Each example: prompt + chosen (good) response + rejected (bad) response from human or AI labels

2

Implicit reward modeling — optimal reward derived from the policy model's own log-probabilities

3

Contrastive loss increases chosen log-prob and decreases rejected log-prob

4

KL penalty against a frozen reference model prevents drift from general capabilities

5

Single supervised training loop — no rollouts, no value function, no iterative RL updates

Practical Details

→ **Data format:** {"prompt": "...", "chosen": "...", "rejected": "..."} — TRL DPOTrainer standard.

→ **Memory cost:** Two model copies in VRAM (policy + frozen reference). QLoRA helps mitigate.

→ **Pipeline position:** Applied after SFT. SFT teaches instruction following; DPO refines quality.

→ **Beta (β):** Controls KL penalty strength. Higher = conservative. Typical range: 0.1–0.5.

→ **When to use:** You have preference data and want alignment with supervised-learning simplicity.

DPO achieves alignment with supervised-learning simplicity — no reward model, no PPO, just preference pairs and a contrastive loss.

3

RLHF – RL from Human Feedback

Core Idea: RLHF is a three-stage pipeline – SFT, reward model training, then PPO optimization – that aligns the model's behavior with human preferences.

The Three Stages

1

Supervised Fine-Tuning

Fine-tune base model on instruction-response pairs to establish instruction-following ability. Produces the initial policy.

2

Reward Model Training

Humans rank responses. A separate model learns to predict which response a human would prefer, outputting a scalar score.

3

PPO Optimization

Policy generates responses → reward model scores them → PPO updates policy to maximize reward with KL penalty against SFT model.

Practical Details

→ **Four models in memory:** Policy, reference (frozen SFT), reward model, and value head – most memory-intensive method.

→ **Training instability:** PPO is sensitive to hyperparameters. Reward hacking is a known failure mode.

→ **Annotation cost:** Stage 2 requires large-scale human preference labeling – expensive and slow.

→ **Historical significance:** The method behind ChatGPT, Claude, and other aligned frontier LLMs.

→ **When to use:** Highest-quality alignment needed, budget for annotation + engineering complexity.

RLHF is the original alignment breakthrough – powerful but complex, requiring four models and careful hyperparameter tuning.

4

ORPO – Odds Ratio Preference Optimization

Core Idea: ORPO combines instruction tuning and preference alignment into a single training pass by adding an odds-ratio penalty directly to the SFT loss.

How It Works

1

Unified loss = SFT cross-entropy on chosen response + odds-ratio preference penalty

2

Odds ratio: $P(\text{response}) / (1 - P(\text{response}))$ – more stable gradients than raw log-probabilities

3

No reference model needed – SFT component naturally anchors coherent instruction following

4

Single-stage training – SFT and preference alignment happen simultaneously in one pass

5

Same data format as DPO: prompt + chosen + rejected. Chosen serves as both SFT target and preference

Practical Details

→ **Memory savings:** No reference model $\approx$ half the VRAM of DPO. Critical difference on consumer GPUs.

→ **Training efficiency:** One run replaces two (SFT + DPO). Halves total training time and pipeline complexity.

→ **Lambda (λ):** Controls preference penalty weight vs SFT loss. Higher = stronger alignment. Range: 0.1–1.0.

→ **Maturity:** Published 2024 – competitive with DPO on benchmarks, but less production validation.

→ **When to use:** Want alignment + SFT in one pass, memory-constrained, or simplifying pipeline.

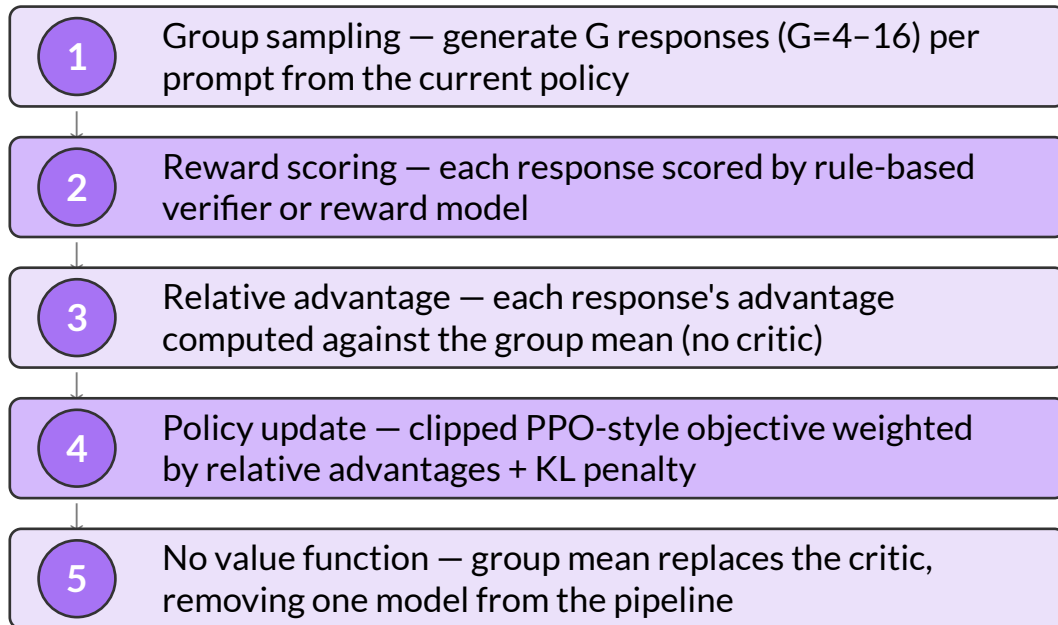
ORPO merges SFT and alignment into one training pass – no reference model, no two-stage pipeline, just one unified objective.

5

GRPO – Group Relative Policy Optimization

Core Idea: GRPO generates multiple responses per prompt, scores them with a reward function, then uses the group's relative ranking to update the policy – no critic network needed.

How It Works



Practical Details

- **DeepSeek R1:** GRPO was used to train DeepSeek-R1, producing strong reasoning without full RLHF.
- **Best with verifiable tasks:** Math (check answer), code (run tests), extraction (validate schema).
- **Compute trade-off:** G forward passes per example, but offset by eliminating the critic network.
- **Group size:** Larger groups = stable estimates but more compute. $G=8$ is a common starting point.
- **When to use:** Verifiable rewards, want RL-style optimization without critic or trained reward model.

GRPO uses group-relative ranking to optimize policy – no critic, no reward model training, just sample, score, and update.

Cost-Benefit Analysis of Fine-Tuning



Making Data-Driven Decisions on Model Customization

Module 7: Model Customization • Section 7.5

Nunnari Academy

Generative AI & Agentic AI Program

What We Will Cover

1

The True Cost of Fine-Tuning

Compute, data, talent, and hidden costs teams often underestimate

2

The Benefits You Gain

Performance, consistency, latency, and competitive advantages

3

When the Math Works

Scenarios where fine-tuning delivers clear ROI

4

When It Doesn't Pay Off

Situations where the investment outweighs the return

The True Cost of Fine-Tuning

1 The True Cost of Fine-Tuning

Key Insight: Fine-tuning costs extend far beyond GPU hours — teams must account for the full lifecycle from data preparation to ongoing maintenance.

1 **Data Preparation** — Collecting, cleaning, and labeling 500–10,000+ examples — weeks to months of expert time

2 **Compute & Infrastructure** — GPU/TPU hours for training, hyperparameter sweeps, and evaluation (\$50 to \$100K+)

3 **Talent & Expertise** — ML engineers to design, train, evaluate, and debug — a specialized and expensive skill set

4 **Evaluation & Iteration** — Multiple training runs to find optimal settings, evaluate regression, and validate quality

5 **Ongoing Maintenance** — Model monitoring, retraining as data drifts, and versioning infrastructure over time

The Benefits You Gain

2

Benefits of Fine-Tuning

Performance Gains

- ✓ Higher accuracy on domain-specific tasks (often 10–30% improvement over prompting)
- ✓ Consistent output format and style across thousands of calls
- ✓ Reduced hallucination within the fine-tuned domain
- ✓ Internalizes patterns that prompts cannot reliably enforce

Operational Gains

- ✓ Lower per-call latency — shorter prompts mean faster responses
- ✓ Reduced token costs at scale — no lengthy system prompts or few-shot examples needed
- ✓ Competitive moat — a fine-tuned model is harder to replicate than a prompt
- ✓ Simplified application code — less prompt engineering and output parsing logic

When the Math Works

3 When Fine-Tuning Pays Off

ROI Trigger: Fine-tuning delivers return when volume, stability, and data quality all align.

- ✓ You make 10,000+ API calls per month — token savings from shorter prompts compound fast
- ✓ Consistency is business-critical — legal, medical, or compliance outputs that must follow strict formats
- ✓ You have a stable, well-defined task — the requirements won't change drastically every quarter
- ✓ Prompt engineering has hit its ceiling — you've iterated extensively and still can't get reliable results
- ✓ Latency is a competitive advantage — shaving 200ms per call matters at scale

Break-even: if prompt token savings × monthly call volume > fine-tuning cost, the investment pays for itself.

When It Doesn't Pay Off

4 When NOT to Fine-Tune

Warning: Fine-tuning is a poor investment when any of the following conditions apply.

-  Your task or domain changes frequently — you'll retrain more often than you benefit
-  You have fewer than 500 high-quality examples — insufficient data leads to overfitting
-  Prompt engineering already achieves 90%+ of your target quality
-  Your team lacks ML expertise — debugging and evaluation cost more than the gains
-  You're still validating product-market fit — the use case may pivot before you recoup the cost

The most expensive mistake is fine-tuning too early — before the task, data, and requirements are stable.

ROI Decision Framework

Factor	Low ROI	High ROI
Call Volume	< 1,000 / month	> 10,000 / month
Task Stability	Changes quarterly	Stable for 6+ months
Data Quality	< 500 noisy examples	1,000+ curated pairs
Prompt Performance	Already 90%+ accuracy	Below 80% accuracy
Team Expertise	No ML engineers	Dedicated ML team
Latency Needs	Not a differentiator	Competitive advantage

More factors in the “High ROI” column = stronger case for fine-tuning.

Key Takeaways

- 1 **Count all costs** — Data prep, talent, iteration, and maintenance often exceed raw compute costs
- 2 **Quantify the benefits** — Measure accuracy gains, token savings, and latency improvements in real numbers
- 3 **High volume is the multiplier** — Fine-tuning ROI scales with the number of calls your system makes
- 4 **Stability is a prerequisite** — Only fine-tune when the task, data, and requirements are locked in
- 5 **Start with prompting** — Always exhaust prompt engineering and RAG before committing to fine-tuning

Fine-tuning is an investment — run the numbers before you run the training job.

Real-World Scenarios



Scenario: Startup with Limited Hardware

The Situation

- Fine-tune a 13B model for customer support
- Single RTX 4090 (24 GB VRAM)
- Minimal budget for cloud GPUs
- Need production-quality results

The Choice: QLoRA

- Load 13B model in NF4 (~6.5 GB)
- Inject LoRA adapters on attention layers
- Train with paged AdamW optimizer
- Total VRAM: ~18 GB — fits on 4090

The Result

- Production-quality support model
- Training cost: electricity only
- Adapter size: ~30 MB
- Can train new adapters for more tasks



Scenario: Enterprise Multi-Task Platform

The Situation

- 200 internal teams, each with a specialized task
- One 70B model on a GPU cluster
- Storage and switching overhead must be minimal

The Choice: Prompt Tuning

- Train a unique soft-prompt per task (~80 KB each)
- All 200 vectors stored in under 16 MB total
- Swap prompts at request time, zero model reload

The Result

- One frozen 70B model serves 200 tasks
- Per-task storage: ~80 KB
- Task switching: instant
- Quality at 70B scale: near full fine-tuning



Scenario: Research Lab

The Situation

- Fine-tune 7–13B models for domain reasoning
- Quality is the top priority
- A100 GPUs (80 GB) available
- Need to compare multiple runs efficiently

The Choice: LoRA

- LoRA rank 32–64 on all linear layers
- Full BFloat16 base (no quantization)
- One adapter per experiment (~20–50 MB)
- Swap adapters for easy A/B comparison

The Result

- Near full fine-tuning quality
- 10× less compute per run
- Small adapter files per experiment
- Reproducible and storage-efficient



Quick Decision Guide

If your priority is...	Choose	Why
Best overall quality	LoRA	Strongest quality-to-efficiency ratio across model sizes
Minimum GPU memory	QLoRA	4-bit quantization + LoRA cuts VRAM by 10–16×
Fewest trainable parameters	Prompt Tuning	<0.01% of model params; ~80 KB per task
Hundreds of tasks, one model	Prompt/Prefix	Negligible per-task storage; instant switching
Stronger adaptation on small models	LoRA / Prefix	Prompt tuning underperforms below 10B
API-only access	Prompt Tuning	Only method that works without weight access
Legacy compatibility	Adapters	Still supported in older codebases

When in doubt, start with LoRA — it covers the widest range of use cases with the fewest surprises.

Key Takeaways

1

No Universal Best: The right PEFT method depends on your GPU budget, model size, task complexity, and deployment needs.

2

LoRA Is the Default: When no special constraint applies, LoRA offers the best balance of quality, efficiency, and tooling support.

3

QLoRA for Tight Budgets: 4-bit quantization makes fine-tuning large models possible on consumer hardware without quality loss.

4

Soft Prompts for Scale: Prompt and prefix tuning shine when serving hundreds of tasks from a single frozen model.

5

Combine When Possible: The strongest pipelines layer methods — QLoRA for training, LoRA for production, prompt tuning for routing.

PEFT is not about choosing one method forever — it's about matching the right tool to the constraint you face today.

Understanding Unsloth



The Theory Behind Efficient LLM Fine-Tuning

Module 4: Model Fine-Tuning • Section 4.1

Nunnari Academy

Generative AI & Agentic AI Program

**What is
Unslowth?**

1 What is Unsloth?

Unsloth: An open-source library that makes fine-tuning large language models dramatically faster and more memory-efficient — without changing the final result.

Background

Created by: Daniel and Michael Han in 2023

Born from: Frustration — fine-tuning a 7B model required expensive A100 GPUs at \$2–3/hour

Goal: Make LLM fine-tuning accessible on consumer-grade hardware

Results: 2–5× faster training and 60–80% less VRAM than standard HuggingFace

Ecosystem: Fully open source, integrates seamlessly into HuggingFace

What it is NOT

✗ Not a new training algorithm — it uses the same math

✗ Not a new model architecture

✗ Not a cloud service or paid platform

✗ Not a replacement for HuggingFace — it wraps and extends it

✗ Not magic — gains come from hand-optimized low-level CUDA kernel rewrites

2

The Problem Before Unsloth

The hardware wall: Standard fine-tuning was gated behind enterprise hardware — a 7B model consumed 40GB+ VRAM just to train.

The Hardware Wall

~40GB VRAM needed for full fine-tune of a 7B model

~18–24GB VRAM even with standard LoRA, before Unsloth

\$2–3/hr Cost of an A100 80GB GPU on cloud platforms

8–16GB VRAM most developers actually own

Inaccessible Fine-tuning was effectively out of reach for individuals and small teams

Why VRAM Fills Up

Model weights: The billions of parameters occupying base memory

Optimizer states: Adam stores momentum + variance — 2× the model size again

Gradients: Backpropagation requires a full copy of every weight's gradient

Activations: Every layer output cached during the forward pass for backprop

Simultaneous: All of the above must live in VRAM at the exact same time during training

The FastLanguageModel Wrapper

1 FastLanguageModel – What It Actually Is

Wrapper: FastLanguageModel sits between you and HuggingFace – transparently replacing slow operations with hand-optimized CUDA kernels while keeping the same familiar API.

What a Wrapper Does

Definition: A class that takes an existing object and replaces or extends its methods

Same API: You interact with HuggingFace exactly as before – nothing about your workflow changes

Internals: Unsloth rewrites the three most expensive operations in Triton/CUDA from scratch

Rewrites: Attention mechanism, cross-entropy loss, and RoPE embeddings – ~70% of training compute

No math change: Same numerical result produced – just computed significantly faster

The Key Methods

from_pretrained() Loads model in 4-bit, patches kernels, returns standard model object

get_peft_model() Attaches LoRA adapters, configures which layers are trainable

for_inference() Swaps in fast generation kernels, disables training-specific overhead

save_pretrained_merged() Dequantizes base, folds in adapter, saves clean 16-bit model

save_pretrained_gguf() Exports to GGUF format for local deployment with llama.cpp

2

Gradient Checkpointing – Unsloth's Secret Weapon

Gradient Checkpointing: Unsloth's custom implementation saves an additional 30% VRAM by intelligently deciding which activations to keep and which to recompute on demand.

The Memory Problem with Backprop

Forward pass: Every layer's output (activations) must be saved for later backpropagation

32-layer model: Simultaneously holds 32 sets of activations in VRAM

Long sequences: Activation memory can exceed savings from quantization itself

Standard checkpointing: Discard all activations, recompute everything during backward – very slow

The dilemma: Store activations (fast but VRAM-heavy) vs recompute (slow but VRAM-light)

Unsloth's Smarter Approach

Selective strategy: Identifies which activations are cheap vs expensive to recompute

Cheap → discard: Simple element-wise ops are dropped and recomputed on demand

Expensive → keep: Attention outputs selectively retained because recomputing them is costly

VRAM saving: 30% additional reduction on top of QLoRA's existing savings

Speed trade-off: Only 5–10% speed penalty – a very favorable exchange

Advantages & Limitations

Why Unsloth Stands Out

Core value: Unsloth delivers enterprise-level fine-tuning capability on consumer hardware — democratizing LLM customization for individuals and small teams.

1 Speed: 2–5× faster training than standard HuggingFace. On a T4 GPU, what took 4 hours now takes under 1 hour

2 Memory: 60–80% VRAM reduction. A 7B model that needed 40GB now fits in 6–8GB — a free Colab T4 GPU

3 Zero accuracy loss: Custom kernels produce mathematically identical results. No quality trade-off for the speed gains

4 HuggingFace integration: Works with HF datasets, tokenizers, Trainer, PEFT, and the entire ecosystem without changes

5 Export flexibility: Models exported as LoRA adapters, merged 16-bit, GGUF Q4/Q8, or pushed directly to HuggingFace Hub

Unsloth makes fine-tuning accessible — not just faster. A student with a free Colab account can now do what previously required a \$500/month GPU server.

When Unsloth Falls Short

Important: Unsloth's optimizations come with specific constraints — understanding them prevents frustration and ensures you use the right tool for each job.

Technical Limitations

Single GPU only: No distributed or multi-GPU training support as of 2024

NVIDIA only: Custom CUDA kernels require an NVIDIA GPU — no AMD, no Apple Silicon

Model whitelist: Only specific supported architectures get the full speed benefit

Quantization drift: 4-bit training introduces slight numerical differences vs full precision

Long context limits: Fine-tuning beyond 8K tokens loses some efficiency advantages

When to Use Something Else

Multi-GPU training: Use HuggingFace Accelerate + DeepSpeed instead

AMD GPUs: Use standard PEFT + bitsandbytes without Unsloth

Maximum quality: Full fine-tuning (not LoRA) may suit tasks requiring absolute precision

Production at scale: Consider full precision merged models for concurrent high-load serving

Continual pretraining: Large-scale pretraining pipelines are better suited than Unsloth's SFT focus

Summary

Key Takeaways

- 1 **Unsloth is a CUDA-optimized wrapper:** around HuggingFace — same API, dramatically faster execution through hand-written kernels
- 2 **QLoRA is the core technique:** 4-bit frozen base model + 16-bit trainable LoRA adapters = 80% less VRAM, same quality
- 3 **FastLanguageModel is the entry point:** it loads, patches, wraps, and exports everything through a clean unified interface
- 4 **The gains are real and lossless:** 2–5× speed, 60–80% memory, zero accuracy drop — benchmarked, not just marketing
- 5 **Know the limits:** Single GPU only, NVIDIA only, specific model support — the right tool for the right job

Fine-tuning is no longer a privilege of large compute budgets — Unsloth puts it in the hands of every developer with a GPU.

Dataset Preparation for Fine-Tuning



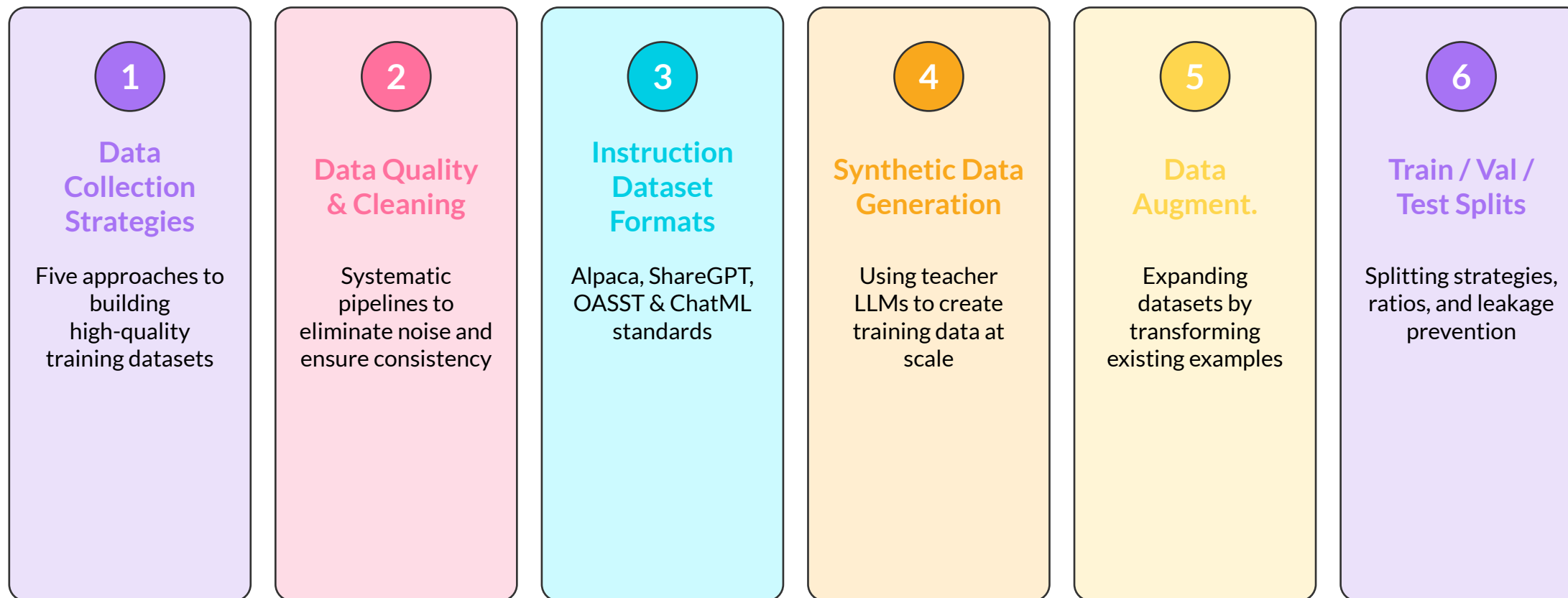
From Collection to Clean, Formatted, Production-Ready Training Data

Module 7: Fine-Tuning LLMs • Section 7.2

Nunnari Academy

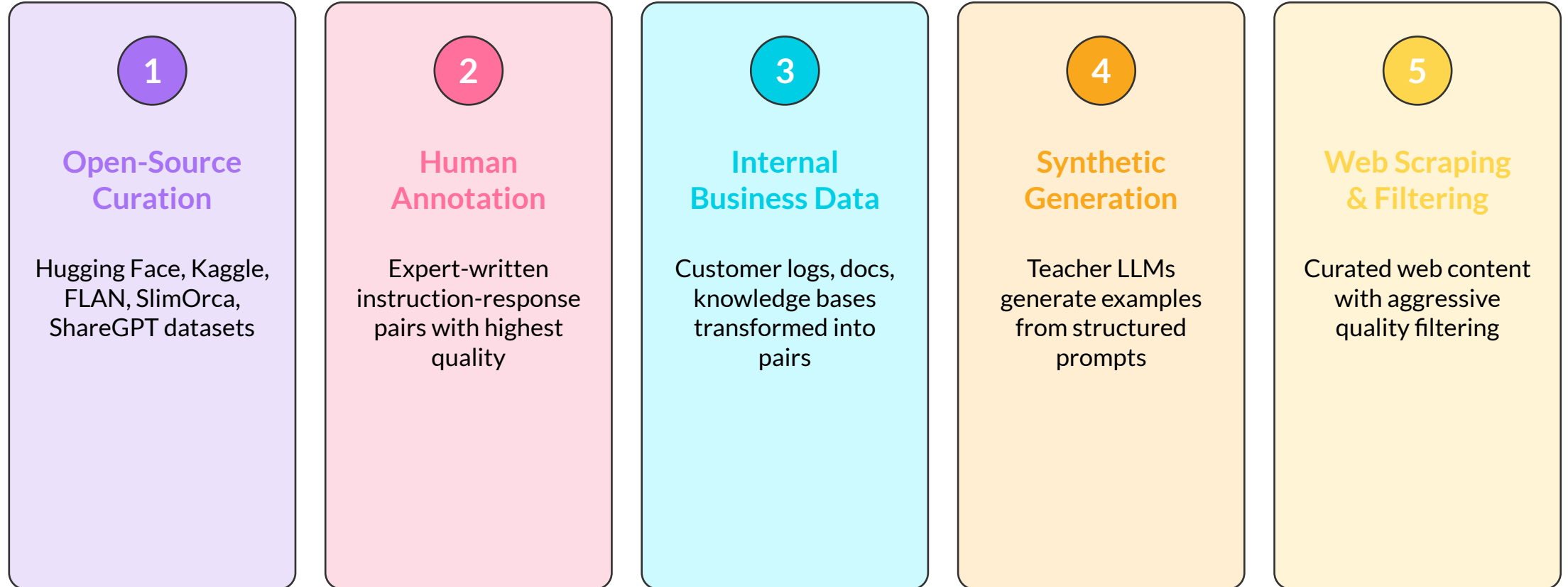
Generative AI & Agentic AI Program

What We'll Cover



Data Collection Strategies

Five Core Collection Strategies





Choosing the Right Strategy

Factor	Open-Source	Human	Internal	Synthetic
Cost	Very Low	High	Medium	Low-Medium
Quality	Variable	Highest	High	Medium-High
Scale	High	Low	Depends	Very High
Domain Specificity	General	Excellent	Excellent	Good
Speed	Fast	Slow	Medium	Fast
IP/License Risk	Check Each	None	Check Policy	Check ToS

No single strategy is sufficient — hybrid approaches combining human expertise with synthetic scale produce the best results.



Recommended Hybrid Pipeline

Five phases that combine the strengths of multiple collection strategies.

1

Define Gold Standards: Write 20–50 expert examples that define quality, format, tone, and depth expectations

2

Curate Open-Source Base: Filter public datasets (FLAN, SlimOrca, ShareGPT) for relevant, high-quality examples

3

Generate Synthetic at Scale: Use teacher LLMs with structured prompts to expand dataset 5–10x

4

Augment with Internal Data: Transform real customer interactions, documentation, and domain corpora into training pairs

5

Refine with Human Review: Sample, rate quality, correct errors, and create targeted examples for weak areas

Quality beats quantity — a clean 1,000-example dataset often outperforms a noisy 50,000-example one.

Data Quality and Cleaning

1 Dimensions of Data Quality

Data quality is multi-dimensional — a dataset can score well on one dimension while failing critically on another.

1 **Accuracy:** Responses are factually correct — inaccurate examples teach confident hallucination

2 **Relevance:** Each example matches the target task — irrelevant pairs dilute focus and waste capacity

3 **Consistency:** Similar instructions receive similar-quality, similarly-formatted responses throughout

4 **Completeness:** Every response fully addresses the instruction — no truncations or skipped parts

5 **Uniqueness:** No excessive duplicates or near-duplicates that cause memorization and bias

6 **Formatting:** Proper structure, encoding, and template compliance across all examples

Fine-tuning amplifies every pattern in your data — including the bad ones.

2 Cleaning Pipeline: Automated Stages

Move from fast, automated checks to slower, more nuanced quality assessments.

1 **Structural Validation:** Verify fields exist, JSON parses, templates are correct — reject malformed entries

2 **Length & Content Filtering:** Remove outliers that are too short (trivial) or too long (will be truncated during training)

3 **Deduplication:** Hash-based exact dedup, then embedding similarity (cosine > 0.85–0.95) for near-duplicates

4 **Language & Encoding Cleanup:** Filter wrong languages, fix mojibake, normalize whitespace, standardize Unicode characters

These four stages are fully automatable and should run on every dataset before any manual review.

3 Cleaning Pipeline: Quality Assessment

Deeper quality assessment combining automated scoring with targeted human review.

1 **Quality Scoring:** LLM-as-judge rates instruction clarity, response accuracy, and helpfulness (1–5 scale)

2 **Safety & PII Filtering:** Toxicity detection, personally identifiable information removal, bias scanning

3 **Manual Review:** Human spot-check 5–10% sample stratified by topic and difficulty — target < 5% error rate

Combine automated tools with targeted human review for maximum coverage.

4

Key Quality Metrics & Targets

Metric	Target	Measurement Method
Dedup Rate	< 3% duplicates remaining	MinHash + embedding similarity
Avg Quality Score	≥ 4.0 / 5.0	LLM-as-judge on random sample
PII Detection Rate	0% PII in final set	Regex + NER model scan
Format Compliance	100% valid structure	Schema validation
Length Distribution	Within 1–2σ of target	Histogram + percentile analysis
Toxicity Rate	< 0.1%	Perspective API / classifier

Track these metrics across every cleaning iteration — data quality is an ongoing process, not a one-time step.

Instruction Dataset Formats

1 Alpaca Format – Single-Turn

Three fields – instruction, input (optional), output – the simplest and most widely adopted format.

```
{  
  "instruction": "Summarize the following  
                 paragraph in one sentence."  
  "input": "The Amazon rainforest  
           covers over 5.5 million sq km..."  
  "output": "The Amazon rainforest is  
           the world's largest tropical  
           forest, housing 10% of species."  
}
```

Single-turn only

One instruction → one response per example

Optional input field

Additional context when the task needs reference data

52K original examples

Stanford's Alpaca dataset set the standard in early 2023

Universal support

Supported by every major fine-tuning framework

2

ShareGPT Format — Multi-Turn

A conversations array with alternating role objects — captures full multi-turn dialogue context.

```
{  
  "conversations": [  
    { "from": "human",  
      "value": "Explain quantum..." },  
    { "from": "gpt",  
      "value": "Quantum computing..." },  
    { "from": "human",  
      "value": "Give me an example?" },  
    { "from": "gpt",  
      "value": "Drug discovery..." }  
  ]  
}
```

Multi-turn support

Preserves full conversation flow with context

Role-based structure

human/gpt/system roles clearly separated

Natural chat behavior

Scraped from real user interactions with ChatGPT

Recommended default

Most expressive format — use as master storage

3

OASST Format – Tree + Rankings

Multiple ranked responses per prompt – built-in preference data for DPO/RLHF training.

Tree Structure

 **Prompt:** "Explain quantum computing"

- **Response A** (rank 0 – best)
 - Follow-up question
 - Follow-up response
- **Response B** (rank 1)
- **Response C** (rank 2 – worst)

Preference data built-in

Ranked responses enable DPO/RLHF without extra labeling

Quality metadata

Human-annotated labels for helpfulness, toxicity, creativity

Multilingual

Conversations in 35+ languages across both v1 and v2

Community-validated

Each response rated by multiple human annotators

4 ChatML – The Unifying Template

Dataset format (storage) → Chat template (tokenization) → Token sequence (training)

Model	Template Style	Special Tokens
LLaMA 3	LLaMA-style	< begin_of_text >, < start_header_id >
Mistral / Mixtral	Mistral-style	[INST], [/INST]
Qwen	ChatML	< im_start >, < im_end >
Phi-3	ChatML variant	< user >, < assistant >, < end >
Gemma	Gemma-style	<start_of_turn>, <end_of_turn>

Mismatched chat templates are a leading cause of degraded fine-tuning results — always verify the template matches your base model.

5

Format Comparison at a Glance

Feature	Alpaca	ShareGPT	OASST
Turn Type	Single-turn	Multi-turn	Multi-turn tree
System Prompt	No (use input field)	Yes (system role)	Yes (implicit)
Preference Data	No	No	Yes (ranked)
Complexity	Simplest	Moderate	Most complex
Best For	Task-specific SFT	Conversational SFT	DPO / RLHF

Store your master dataset in ShareGPT format (most expressive) – convert to other formats as needed.

Synthetic Data Generation

1 Core Generation Techniques

Self-Instruct

- 1 Start with 175 seed instructions
- ↓
- 2 LLM generates new instructions
- ↓
- 3 Classify: generative or classification
- ↓
- 4 LLM generates responses
- ↓
- 5 Filter low-quality & duplicates

Evol-Instruct

- 1 Add constraints to narrow scope
- ↓
- 2 Deepen reasoning requirements
- ↓
- 3 Concretize with specific details
- ↓
- 4 Increase multi-step reasoning
- ↓
- 5 Broaden to adjacent topics

2 Prompt Engineering for Data Generation

The quality of synthetic data is directly determined by the quality of your generation prompts.

- 1 **System Prompt Design:** Specify role, quality standards, format requirements, and constraints for the teacher model
- 2 **Diversity Injection:** Topic lists, difficulty levels, persona randomization, temperature variation to prevent homogeneity
- 3 **Few-Shot Seeding:** 3–8 carefully chosen examples per prompt demonstrating exact format, depth, and quality expected
- 4 **Chain-of-Thought:** Ask teacher to include reasoning — teaches student model how to think, not just what to answer
- 5 **Negative Examples:** Generate deliberately incorrect responses with corrections — produces preference data for DPO

Without explicit diversity mechanisms, 10,000 examples might cover only 50 topics phrased in 3 ways.

3 Quality Control for Synthetic Data

Synthetic data's greatest risk: quality problems are invisible individually but devastating at dataset scale.

1 **Automated Filtering:** Format validation, length filtering, language detection, deduplication, consistency checking

2 **LLM-as-Judge:** Different model evaluates instruction clarity, response accuracy – avoids self-confirming quality

3 **Human-in-the-Loop:** Review 5–10% random sample stratified by topic, difficulty, and batch – target < 5% error rate

4 **Factual Verification:** Cross-reference claims against sources; run code examples; verify against knowledge bases

5 **Diversity Auditing:** Cluster embeddings to verify topic coverage; calculate pairwise distance; fill coverage gaps

Use a different model as judge than the teacher to avoid self-confirming quality errors.

4 Critical Risks of Synthetic Data

- 1 **Model Collapse:** Training on synthetic outputs of previous synthetic outputs progressively degrades quality
- 2 **Quality Ceiling:** Student model can never exceed teacher quality – synthetic-only training has inherent limits
- 3 **Stylistic Homogeneity:** Teacher LLMs produce recognizable patterns – 100% synthetic data creates monotone outputs
- 4 **Licensing Compliance:** Some model providers prohibit using outputs to train competing models – check ToS carefully
- 5 **Evaluation Contamination:** Synthetic test data may share distribution with training data, inflating performance metrics

Always include human-written data in your training mix to prevent model collapse and stylistic homogeneity.

Data Augmentation Techniques

1 Instruction-Level Augmentation

Modify the user-facing prompt while keeping the core response intact — builds robustness to phrasing variability.

1 **Paraphrasing:** Rewrite instructions preserving meaning — same question asked in 3–5 different phrasings

2 **Complexity Scaling:** Simplify for beginners and complexify for experts — same concept at multiple difficulty levels

3 **Format Variation:** Rephrase as question, command, fill-in-the-blank, or scenario-based prompt

4 **Persona Injection:** Add context: “as a data scientist”, “for a 10-year-old”, “in a business context”

5 **Constraint Addition:** Add length limits, format requirements, or style constraints to existing instructions

Same underlying question asked many different ways — builds robustness to real-world phrasing variability.

2 Response-Level Augmentation

Keep the instruction fixed, vary the response to teach multiple valid answer styles.

1 **Style Transfer:** Formal → casual → bullet-point → step-by-step → analogy-based explanation

2 **Length Variation:** Compress 300-word answer to 50-word summary and expand to 500-word deep dive

3 **Explanation Reframing:** Same concept: mathematical, real-world analogy, code example, step-by-step walkthrough

4 **Multi-Format Output:** Explanation → comparison table → flowchart description → Q&A format → JSON

5 **Refinement Chains:** Draft → corrected → polished versions create preference data for DPO training

Response augmentation teaches the model that multiple valid answer styles exist for each question.

3

LLM-Powered Augmentation

Higher-quality than rule-based methods — LLMs understand semantic meaning during transformation.

1

Back-Translation

EN→FR→EN creates natural paraphrases preserving meaning

2

LLM Paraphrasing

Direct model-based rewrites with controlled diversity

3

Counterfactual Generation

Change key variables: what if the input had different parameters?

4

Difficulty Transformation

Scale examples up/down in complexity while keeping core concept

5

Domain Transfer

Adapt examples from one domain to another with context mapping

Limit augmentation to 3–5x original dataset size to avoid overfitting on augmentation artifacts.

Train / Validation / Test Splits

Understanding the Three Sets

Training Set

70-90%

The data the model learns from.
Determines knowledge, style, and output format.

- ✓ Largest split
- ✓ Directly influences behavior
- ✓ Used for gradient updates

Validation Set

10-15%

Monitors performance during training.
Never used for gradient updates.

- ✓ Detects overfitting
- ✓ Selects best checkpoint
- ✓ Tunes hyperparameters

Test Set

10-15%

Final unbiased evaluation after all training is complete.

- ✓ Used only once at end
- ✓ Unbiased performance estimate
- ✓ Must be human-validated

Using the test set for any purpose other than final evaluation contaminates your performance estimates.

1 Recommended Split Ratios

Optimal ratios depend on total dataset size — LLM fine-tuning datasets need different ratios than traditional ML.

Dataset Size	Train	Validation	Test	Notes
50K+	90%	5%	5%	Comfortable margins for all sets
5K–50K	80–85%	10%	5–10%	Standard for most fine-tuning
1K–5K	80%	10%	10%	Min 100–200 test examples
< 1K	70–80%	10–15%	10–15%	Min 50 val + 50–100 test

Even with only 500 examples, reserve at least 50 for validation and 50 for test.

2 Splitting Strategies

How you split matters as much as how much — the wrong strategy creates hidden data leakage.

1 **Random Split:** Simple random assignment — works only when examples are truly independent (no shared templates)

2 **Stratified Split:** Maintain proportional representation of task types, difficulty levels, and categories across all sets

3 **Group-Based Split:** Keep conversation families, template groups, and source documents together in the same set

4 **Temporal Split:** Train on older data, test on newer — simulates real-world deployment where future data differs

5 **Topic-Holdout:** Reserve entire topic categories for testing — measures genuine generalization to unseen domains

Use stratified or group-based splitting to prevent hidden leakage between train and test sets.

3

Preventing Data Leakage

Data leakage is the most common cause of over-optimistic fine-tuning results.

1

Augmentation Leakage: Augmented copies of training examples end up in test set — always augment AFTER splitting

2

Template Leakage: Same template generates examples in both sets — split by template group, not individual examples

3

Conversation Leakage: Turns from the same conversation appear in different sets — split conversations as atomic units

4

Semantic Leakage: Near-duplicate meanings across sets — check embedding similarity between train and test

5

Source Leakage: Same source document generates examples in both sets — track provenance and split by source

Split before augmenting, keep families together, and check for semantic duplicates across splits.

Key Takeaways

Dataset Preparation — Key Takeaways

1 **Quality Over Quantity:** A clean 1,000-example dataset often outperforms a noisy 50,000-example one in fine-tuning

2 **Hybrid Collection:** Combine open-source curation, synthetic generation, and human review for best results

3 **Format Matters:** Match your dataset format to the model's chat template — mismatches silently degrade performance

4 **Systematic Cleaning:** Build a reproducible pipeline: structural checks → dedup → quality scoring → manual review

5 **Split with Care:** Stratified/group-based splits, augment only training data, and lock the test set permanently

Dataset preparation is not preprocessing — it is the single highest-leverage activity in fine-tuning.

Thank You!



Nunnari Academy

Generative AI & Agentic AI Program